

TUGAS MENJELASKAN PROGRAM

Tugas Ini Dibuat Guna Memenuhi Mata Kuliah Struktur Data



Dosen pengampu:

Adam bachtiar, s.kom, M.MT

Disusun Oleh :

Nama : nurul amalia prasiska

NIM : 24241074

Kelas : PTI B

**PROGRAM STUDI PENDIDIKAN TEKNOLOGI INFORMASI
FAKULTAS SAINS, TEKNIK DAN TERAPAN
UNIVERSITAS PENDIDIKAN MANDALIKA MATARAM
2025**

❖ Membuat node

```
1 # Node sebagai elemen linked list
2 def buat_node(data):
3     return {"data": data, "next": None}
4
```

Penjelasan:

1. Fungsi ini digunakan untuk membuat satu node dalam linked list.
2. Node diwakili dengan dictionary yang memiliki dua elemen:
 - "data" → untuk menyimpan nilai elemen stack.
 - "next" → untuk menunjuk ke node berikutnya (default None).

❖ Menambah elemen ke stack

```
5 # Fungsi-fungsi stack
6 def push(stack, data, kapasitas):
7     if ukuran(stack) >= kapasitas:
8         print("Stack sudah penuh!")
9         return stack
10    node_baru = buat_node(data)
11    node_baru["next"] = stack
12    stack = node_baru
13    tampilkan_stack(stack)
14    return stack
15
```

Penjelasan:

1. Mengecek apakah ukuran stack sudah mencapai kapasitas.
2. Jika sudah penuh, maka elemen tidak ditambahkan.
3. Membuat node baru.
4. Node baru akan menunjuk ke stack lama (elemen sebelumnya).
5. Puncak stack sekarang adalah node baru tersebut.
6. Menampilkan isi stack setelah ditambahkan, lalu mengembalikannya.

❖ Menghapus elemen dari stack

```
16 def pop(stack):
17     if stack is None:
18         print("Stack kosong!")
19         return stack
20     print(f"Elemen {stack['data']} dihapus dari stack.")
21     stack = stack["next"]
22     tampilkan_stack(stack)
23     return stack
24
```

Penjelasan:

1. Mengecek apakah stack kosong.
2. Jika kosong, tampilkan pesan dan kembalikan stack.
3. Menampilkan elemen yang dihapus.
4. Pindahkan puncak stack ke elemen berikutnya (next).
5. Menampilkan stack setelah di-pop dan mengembalikan stack.

❖ Menghitung jumlah elemen dalam stack

```

25 def ukuran(stack):
26     count = 0
27     while stack is not None:
28         count += 1
29         stack = stack["next"]
30     return count

```

Penjelasan:

1. Mengiterasi setiap node dari puncak ke bawah.
2. Menambahkan 1 setiap kali ada node hingga mencapai None.
3. Mengembalikan total jumlah elemen dalam stack.

❖ Menampilkan data paling atas

```

32 def puncak(stack):
33     if stack is None:
34         print("Stack kosong!")
35     else:
36         print(f"Elemen puncak stack: {stack['data']}")
37

```

Penjelasan:

Menampilkan data yang ada di puncak stack (jika tidak kosong).

❖ Fungsi cek penuh

```

38 def cek_penuh(stack, kapasitas):
39     if ukuran(stack) >= kapasitas:
40         print("Stack penuh!")
41     else:
42         print("Stack belum penuh.")
43

```

Penjelasan:

Mengecek apakah jumlah elemen dalam stack sudah mencapai kapasitas.

❖ Menampilkan Isi Stack

```

44 def tampilkan_stack(stack):
45     elemen = []
46     while stack is not None:
47         elemen.append(stack["data"])
48         stack = stack["next"]
49     elemen.reverse()
50     print("Stack :", elemen)
51

```

Penjelasan:

1. Mengambil semua elemen dari puncak ke dasar dan memasukkannya ke dalam list elemen.
2. Kemudian dibalik (reverse) agar ditampilkan dari bawah ke atas (seperti tampilan stack sebenarnya).
3. Misal, jika puncak 30, lalu 20, lalu 10, maka ditampilkan [10, 20, 30].

❖ Fungsi utama

```

52 # Program utama
53 def main():
54     kapasitas = int(input("Tentukan berapa kapasitas stack : "))
55     stack = None
56
57     while True:
58         print("\nPilih menu berikut ini : ")
59         print("1. Menambah isi stack")
60         print("2. Menghapus isi stack")
61         print("3. Cek Ukuran Stack saat ini")
62         print("4. Cek Puncak Stack")
63         print("5. Cek Stack Full")
64         print("6. Keluar")
65         pilihan = input("\nMasukkan pilihan anda : ")
66
67         if pilihan == '1':
68             while True:
69                 data = input("\nMasukkan isi stack : ")
70                 stack = push(stack, data, kapasitas)
71                 lanjut = input("\nMenambah isi Stack Pilih [Ya/Tidak] : ")
72                 if lanjut.lower() != 'ya':
73                     break
74

```

```

75         elif pilihan == '2':
76             stack = pop(stack)
77
78         elif pilihan == '3':
79             print(f"Ukuran stack saat ini: {ukuran(stack)}")
80
81         elif pilihan == '4':
82             puncak(stack)
83
84         elif pilihan == '5':
85             cek_penuh(stack, kapasitas)
86
87         elif pilihan == '6':
88             print("Keluar dari program.")
89             break
90
91         else:
92             print("Pilihan tidak valid!")
93

```

Penjelasan:

1. Meminta pengguna menentukan kapasitas maksimum stack.
2. Stack diinisialisasi dengan None (kosong).
3. Perulangan menu menampilkan menu ke pengguna dan menunggu input pilihan.
4. Push, Pengguna bisa menambahkan lebih dari satu data secara berurutan, hingga menjawab "Tidak".
5. Pop, Menghapus elemen paling atas dari stack.
6. Ukuran, Menampilkan jumlah elemen dalam stack.
7. Puncak, Menampilkan data di puncak stack.
8. Cek penuh, Mengecek apakah stack sudah penuh.
9. Keluar dari program.
10. Menangani input yang tidak sesuai pilihan menu.

❖ Menjalankan Program

```

94 # Jalankan program
95 if __name__ == "__main__":
96     main()

```

Penjelasan:

Menjalankan Program

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\PENYIMPANAN DATA\Struktur Data\modul 3> & C:/Users/user/AppData/Local/Programs/Python/Python38-32/Python.exe C:/Users/user/AppData/Local/Programs/Python/Python38-32/Python38-32\python.exe C:\PENYIMPANAN DATA\Struktur Data\modul 3\tugasm modul-3.py
Tentukan berapa kapasitas stack : 5

Pilih menu berikut ini :
1. Menambah isi stack
2. Menghapus isi stack
3. Cek Ukuran Stack saat ini
4. Cek Puncak Stack
5. Cek Stack Full
6. Keluar

Masukkan pilihan anda : 1

Masukkan isi stack : 11
Stack : ['11']

Menambah isi Stack Pilih [Ya/Tidak] : ya

Masukkan isi stack : 12
Stack : ['11', '12']
```